

Bandit-Based Multi-Agent Monitoring: Two Extensions

Master's research summary, Madhavan S, Dept. of EECE, IIT Dharwad

This document is a plain-language summary of two research papers written during my master's work, built on top of the bandit-based monitoring framework proposed by Nayak et al. (ACC 2024). Both papers are currently being prepared for conference submission and have not been submitted yet, so I'm sharing a summary here rather than the full manuscripts. Feel free to reach out if you'd like more detail.

Background: the problem both papers build on

The starting point for both papers is a monitoring setup where a team of drones and charging stations has to search a grid environment and figure out which cells are 'interesting' and which aren't, using noisy sensor readings. Nayak et al.'s original framework treats this as a multi-armed bandit problem: every grid cell is an arm, and a UCB-style scoring rule decides which cells the team should visit next. A low-level planner then works out actual flight paths under battery and motion constraints. It's a solid framework with good theoretical guarantees, but it treats every cell the same way, with no notion of scale or of which cells are more likely to matter. That's the gap both of my papers try to close, each from a different angle.

Paper 1: Sparsity-Aware and Spatially-Guided Bandit Monitoring

The first paper starts from two observations about how monitoring problems actually look in practice. First, interesting regions tend to cluster rather than show up as scattered single cells, a fire spreads outward, a disease moves through neighboring crops, a gas leak disperses from one source. Second, interesting cells are usually a small fraction of the whole grid, so spending equal effort ruling out every uninteresting cell is wasted work once you already know roughly how many interesting cells exist.

I made two changes to the high-level planner to take advantage of this:

- **A spatial clustering boost.** The acquisition score for each cell gets an added term based on what fraction of its neighbors are already confirmed interesting. Once the team finds one member of a cluster, the score naturally pulls it toward the rest of that cluster instead of wandering off to unrelated parts of the grid.
- **A sparsity-aware stopping rule.** If you know roughly how many interesting cells exist in advance (a reasonable assumption in a lot of real deployments, like a rough fire size estimate), the algorithm can stop as soon as it has found that many confirmed cells, instead of grinding through every remaining cell just to prove they're uninteresting.

Both changes only affect which cells get picked each epoch, not the actual keep or reject rule, so the original anytime correctness guarantee from Nayak et al. carries over without modification. I also worked out a tighter termination bound that reflects the smaller effective search space. Across 50 trials each on small, medium, and large grid scenarios, this brought epoch counts down by roughly 33%, 50%, and 60% respectively compared to the baseline, with no drop in classification accuracy. The reduction gets bigger as the grid gets larger and sparser, which makes sense since there's proportionally more wasted effort to

cut out.

Paper 2: Hierarchical Bandit Monitoring with Nested Grid Refinement

The second paper tackles a different limitation: the original framework assumes every grid cell is a single indivisible unit. In a lot of real scenarios that's not true. A drone can classify an orchard block from altitude, but figuring out whether one particular tree is ready for harvest needs a closer look. A storage zone can be flagged as risky from a distance, but finding the actual leak source needs a low-altitude sweep. The scale at which a drone can efficiently move around and the scale at which the thing you're actually looking for varies are often different, and the flat single-resolution grid can't capture that.

To address this, I restructured the search space into a two-level nested grid: coarse cells, each split into a set of fine cells underneath. Two UCB bandits run together, one at each level. The coarse bandit decides which regions are worth a closer look each epoch, and for each region it selects, a fine bandit decides which subregions inside it are worth visiting. The classification rule is deliberately asymmetric: a coarse cell gets marked as interesting the moment a single fine cell inside it clears the threshold, but it can only be marked as uninteresting once every fine cell inside it has been checked and the combined uncertainty is low enough. This mirrors the actual logic of the problem, a region is interesting if any part of it is, so confirming that only takes one solid piece of evidence, but ruling a region out means checking all of it.

I derived a per-cell complexity term and a finite termination bound for this hierarchical setting that carries over the structure of the original framework's termination proof, and extended the cost accounting to cover both coarse-level travel and the fine-level sensing passes. On a small scenario where I could run both the hierarchical version and an equivalent flat-grid baseline side by side, the hierarchical planner finished in a median of about 87.5 epochs versus roughly 430 for the flat baseline, a 79% drop, with identical accuracy. I also tested medium and large scenarios (up to 400 coarse cells with 9 fine cells each) to check that the approach holds up as the grid scales, and classification stayed perfect across all of them.

Why these fit together

Both papers start from the same baseline and modify the same part of the system, the high-level planner that decides where to look next, while leaving the low-level motion planning and the core UCB machinery intact. One paper adds structure across scale (coarse versus fine), the other adds structure across space and prior knowledge (clustering and sparsity). In both cases the goal was the same: keep the theoretical guarantees of the original framework intact while cutting down the practical cost of running it.

Status

Both papers are complete and currently being prepared for conference submission. I'll link the full PDFs and code here once they've gone through review, or once I'm able to post preprints without conflicting with a venue's dual-submission policy.

Note: this summary paraphrases my own unpublished work for a personal portfolio and does not include the full manuscript text.